

Praktik hardening vulnerable ke hardened

Database Security in Practice
Dendi Zuckergates | Panduan kode, konfigurasi, dan verifikasi

Panduan ini menjelaskan **bagian yang diperbaiki, bentuk kode sebelum dan sesudah, serta cara membuktikan hasilnya**. Gunakan bersama panduan lima simulasi serangan untuk melihat hubungan antara celah dan kontrolnya.

Cara memakai lab yang tersedia

Tiga checkpoint sudah berisi kode sebelum dan sesudah perbaikan. Praktiknya: **buka kode sumber, bandingkan cabangnya, jalankan versi terkait, lalu uji hasil**. Potongan kode dalam panduan adalah referensi perubahan, bukan file lengkap yang harus ditempel menimpa app.py. Panduan ini tidak membuat proyek latihan baru.

Peta perubahan

Tahap	Perubahan	Hal.
1. Query-fixed	Pisahkan input dari perintah SQL.	3
2. Otorisasi objek	Periksa pemilik invoice pada server.	4
3. Hak akun	Gunakan app_runtime dengan hak terbatas.	5
4. TLS dan sumber	Verifikasi server; batasi sumber dengan HBA.	6-7
5. Kredensial	Baca secret dari file dan uji rotasi.	8
6. Error dan log	Respons umum; request ID untuk penelusuran.	9
7. Pemulihan	Backup, restore, dan uji aplikasi hasil restore.	10
8. Pembuktian	Retest, rebuild bila mengedit, batas produksi.	11-12

Urutan berpikir: buktikan masalah pada baseline, pahami perubahan, ulang input yang sama pada versi perbaikan, lalu cek bahwa pengguna sah tetap bisa bekerja.

Seluruh contoh hanya untuk Invoice Lab lokal dengan data sintetis. Password demo bukan kredensial produksi. Lab dan isi kode sumber tidak diubah oleh pembuatan panduan ini.

Persiapan dan peta berkas

Semua perintah PowerShell dijalankan dari folder lab.

1. Jalankan **JALANKAN-LAB.cmd** dari folder kegiatan, atau buka PowerShell di folder lab dan jalankan:

```
.\lab.ps1 Start
```

2. Buka tiga alamat di bawah. Login **alice / Lab-A-2026!** pada tiap versi. Alice memiliki INV-001 dan INV-003; Bob memiliki INV-002.

Versi	Alamat	Ciri utama
Vulnerable	localhost:8081	Query rentan, tanpa cek pemilik detail.
Query-fixed	localhost:8082	Query aman; akses detail masih terbuka.
Hardened	localhost:8083	Cek pemilik, role terbatas, TLS, error umum.

Buka berkas ini di editor

Berkas di lab/	Bagian yang dipelajari
app.py	search(), detail(), connect(), database_error(), audit().
setup_lab.py	Role, schema, grants, sertifikat, dan secret awal.
compose.yaml	LAB_MODE, mount secret, port dan jaringan.
db/pg_hba.conf	Izin role/sumber dan penolakan koneksi.
db/Dockerfile; db/start.sh	TLS server serta izin file private key.
tests/; scripts/backup_restore.sh	Bukti penolakan, rotasi, dan pemulihan.

Sudah ada sejak baseline

Login, sesi bertanda tangan, CSRF untuk POST, beberapa header keamanan, pembatasan panjang pencarian, port loopback, dan transaksi web read-only sudah aktif pada semua checkpoint. Jangan menganggap semuanya baru ditambahkan di hardened. Aturan HBA dan role juga sudah disiapkan saat setup.

LAB_MODE memilih cabang perilaku di app.py. HARD bernilai true hanya pada mode hardened. Ketiga versi memakai dataset yang sama.

1 / Perbaiki query pencarian

app.py: search() | Slide 25 dan 30 | Vulnerable menuju query-fixed

Masalah: q dari browser disambungkan ke SQL. Input dapat ikut mengubah struktur query dan mengganggu batas pelanggan.

Sebelum: nilai disisipkan ke perintah

```
query = (  
    f"SELECT {columns} FROM billing.invoices "  
    f"WHERE customer_id = '{customer}' "  
    f"AND description ILIKE '%{q}%' ORDER BY id"  
)  
rows = conn.execute(query).fetchall()
```

Sesudah: nilai dikirim terpisah

```
query = (  
    f"SELECT {columns} FROM billing.invoices "  
    "WHERE customer_id = %s "  
    "AND description ILIKE %s ORDER BY id"  
)  
rows = conn.execute(query, (customer, "%" + q + "%")).fetchall()
```

Buka cabang **else** pada search(). Query-fixed dan hardened memakai pola kedua. Placeholder %s tidak diberi tanda petik sendiri. **columns adalah daftar kolom konstan di kode**, bukan input dari browser.

Praktik pembuktian

1. Sebagai Alice, cari input di bawah pada 8081, lalu 8082 dan 8083. Tambahkan satu spasi setelah dua tanda minus saat menyalin.

```
' OR '1'='1' --
```

2. Target: 8081 menampilkan 3 invoice; 8082 dan 8083 menampilkan 0 untuk payload ini.

3. Cari **O'Reilly** pada 8082/8083: INV-003 harus muncul. Pencarian kosong tetap menampilkan dua invoice Alice.

Perbaiki yang dibuktikan

Input menjadi nilai pencarian. Validasi panjang input tetap berguna, tetapi tidak menggantikan binding. Nama tabel, kolom, dan arah pengurutan dinamis memerlukan penanganan tersendiri; placeholder nilai bukan pengganti sembarang struktur SQL.

2 / Periksa pemilik invoice

app.py: detail() | Slide 26 dan 31 | Menutup IDOR

Masalah: query detail sudah memakai parameter, tetapi hanya memeriksa nomor invoice. Pengguna yang login masih bisa meminta invoice pelanggan lain.

Sebelum: hanya ID objek

```
row = conn.execute(
    "SELECT * FROM billing.invoices WHERE id = %s",
    (invoice_id,)
).fetchone()
```

Sesudah: ID objek dan pemilik dari sesi

```
row = conn.execute(
    "SELECT * FROM billing.invoices "
    "WHERE id = %s AND customer_id = %s",
    (invoice_id, session["customer"])
).fetchone()
```

Buka cabang **if HARD** pada detail(). Endpoint memakai decorator authenticated. Identitas pelanggan berasal dari sesi yang diverifikasi server; jangan menggantinya dengan customer ID bebas dari browser.

Praktik pembuktian

1. Login Alice pada 8082, lalu minta detail **INV-002**. Data Bob masih terbuka: perbaikan query pencarian belum menyelesaikan otorisasi detail.
2. Login Alice pada 8083 dan minta INV-002. Target: **404**, tanpa data invoice Bob.
3. Minta **INV-001** sebagai Alice di 8083. Target: **200**. Login Bob pada sesi terpisah untuk memastikan INV-002 miliknya tetap dapat dibuka.

Perbaikan yang dibuktikan

Login mengenali pengguna. Otorisasi menentukan objek yang boleh dia akses. Respons 404 merupakan pilihan lab untuk menyamarkan keberadaan objek; hasil utama yang diperiksa adalah data pihak lain tidak diberikan, sementara data sendiri tetap tersedia.

Di aplikasi lain, aturan kepemilikan juga perlu diterapkan pada pencarian, ekspor, unduhan, dan fitur lain yang menyediakan data.

3 / Batasi hak akun aplikasi

setup_lab.py, app.py: connect(), compose.yaml | Slide 27 dan 32

Sebelum: vulnerable dan query-fixed memakai lab_admin, sebuah superuser. **Sesudah:** hardened memakai app_runtime. Pemilik objek adalah lab_owner, role NOLOGIN yang terpisah dari akun aplikasi.

Hak yang diberikan saat setup

```
GRANT CONNECT ON DATABASE invoice_lab TO app_runtime;  
GRANT USAGE ON SCHEMA billing TO app_runtime;  
GRANT SELECT ON billing.invoices TO app_runtime;
```

Role app_runtime dibuat dengan LOGIN, NOSUPERUSER, NOCREATEDB, NOCREATEROLE, dan NOINHERIT. Setup juga mencabut akses database dari PUBLIC dan CREATE pada schema public. Role runtime tidak menjadi pemilik tabel atau anggota role lain.

SQL referensi, sudah diterapkan

Baca blok seed pada **setup_lab.py**. Jangan menjalankan ulang CREATE ROLE atau mengedit runtime/init untuk mengubah database yang sudah ada. Init SQL hanya dijalankan pada inisialisasi volume kosong. Jangan tampilkan file seed hasil generate karena mengandung password.

Periksa hak tanpa membuka secret

```
docker compose exec db psql -U postgres -d invoice_lab -c "\du"  
docker compose exec db psql -U postgres -d invoice_lab -c "\dp billing.invoices"
```

Cari app_runtime dan hak SELECT pada invoice. Periksa juga fungsi connect(): role yang dipakai harus app_runtime saat HARD bernilai true, dengan mount runtime_current pada service hardened.

Buktikan penolakan operasi

```
docker compose --profile tools run --rm tester python tests/attacks.py --scenario S3
```

Target: akun admin bisa membaca internal dan UPDATE sementara lalu rollback. Runtime ditolak **42501** untuk kedua operasi; SELECT invoice tetap berhasil. Suite Test juga memeriksa DELETE, CREATE, atribut admin, dan keanggotaan role.

Read-only belum memisahkan pelanggan

SELECT tingkat tabel masih dapat membaca seluruh invoice. Isolasi Alice dan Bob dilakukan aplikasi. RLS belum diterapkan. default_transaction_read_only pada web bukan pengganti pembatasan hak akun.

4 / Aktifkan verifikasi TLS

app.py: connect(), db/Dockerfile, db/start.sh | Slide 28 dan 39

Sebelum: koneksi dua versi awal memakai sslmode=disable. **Sesudah:** hardened memakai verify-full agar klien memeriksa CA dan nama server selain mengenkripsi koneksi.

Bagian koneksi setelah hardening

```
psycopg.connect(  
    host="db", dbname="invoice_lab", user="app_runtime",  
    password=Path(os.environ["DB_PASSWORD_FILE"]).read_text().strip(),  
    sslmode="verify-full", sslrootcert="/certs/ca.crt",  
    connect_timeout=3, row_factory=dict_row,  
    options="-c statement_timeout=3000 -c default_transaction_read_only=on"  
)
```

Ini bentuk cabang hardened dari connect(). Import dan pengaturan lain tetap mengikuti app.py. Secret koneksi dijelaskan di halaman 8.

Periksa pasangan server dan klien

Bagian	Yang sudah disiapkan lab
setup_lab.py	CA lokal dan sertifikat server dengan nama SAN db.
db/Dockerfile	ssl=on; lokasi server.crt dan server.key; autentikasi SCRAM.
db/start.sh	Private key dimiliki postgres dengan izin 600.
compose.yaml	CA publik di-mount ke /certs; private key tidak di-mount ke aplikasi.

Praktik pembuktian

1. Login pada 8083, lalu buka **http://localhost:8083/api/connection** di browser yang sama. Target: role app_runtime dan ssl true.
2. Buka endpoint yang sama pada 8081: role lab_admin dan ssl false sebagai pembandingan baseline.
3. Pada retest lengkap halaman 11, periksa tes CA salah, hostname salah, dan koneksi plaintext. Ketiganya harus ditolak sesuai penyebab masing-masing.

Cakupan TLS

TLS ini melindungi aplikasi ke PostgreSQL. Browser masih memakai HTTP loopback. Jangan mematikan verifikasi sertifikat hanya agar error hilang; periksa CA, hostname db, dan masa berlaku sertifikat.

5 / Batasi sumber koneksi

db/pg_hba.conf dan compose.yaml | Slide 33 | Membuktikan S4

HBA sudah membatasi sumber sejak setup. Yang dipelajari di sini adalah aturan untuk jalur hardened, dibandingkan dengan pengecualian baseline. Aturan dibaca berurutan; kecocokan pertama menentukan penanganan koneksi.

Aturan inti untuk akun runtime

```
hostssl invoice_lab app_runtime 172.30.26.22/32 scram-sha-256
hostssl invoice_lab app_runtime 172.30.26.40/32 scram-sha-256
# ... aturan tester admin dan database restore ...
host all all 0.0.0.0/0 reject
host all all ::/0 reject
```

Ini cuplikan, bukan pengganti seluruh file. .22 adalah hardened, .40 adalah tester, dan .30 adalah untrusted. Pengecualian lab_admin pada .20/.21 dipertahankan agar dua baseline tetap dapat didemonkan.

Periksa konfigurasi yang dibaca PostgreSQL

```
docker compose exec db psql -U postgres -d invoice_lab -c "SHOW hba_file;"
```

Di compose.yaml, database tidak mempunyai published port. Jaringan backend bersifat internal. Port web hanya dipetakan ke 127.0.0.1. Pengaturan ini sudah ada pada lab, bukan perubahan firewall selama demo.

Praktik pembuktian

1. Gunakan hasil S3: tester .40 berhasil SELECT dengan akun runtime. Ini pembandingan koneksi yang diizinkan.
2. Coba sumber .30 dengan kredensial valid:

```
docker compose --profile tools run --rm untrusted
Get-Content .\evidence\network.json
```

3. Target: pesan penolakan pg_hba.conf terhadap **172.30.26.30**. Timeout atau password salah bukan bukti kontrol HBA yang dimaksud.

Jika mengedit HBA pada latihan tersendiri

Validasi sintaks dan periksa aturan sumber sebelum reload. Jangan menambahkan allow-all. Untuk alur perbandingan dalam panduan ini, tidak perlu mengedit atau me-reload HBA; gunakan konfigurasi paket yang sudah tersedia.

6 / Kelola secret dan rotasi

compose.yaml, app.py, tests/attacks.py | Slide 34 | Membuktikan S5

Pembacaan password dari file sudah digunakan semua checkpoint. Perubahan penting pada hardened adalah memilih **secret milik app_runtime**, bukan secret lab_admin, lalu membuktikan rotasi tetap menjaga akses aplikasi.

Konfigurasi service hardened yang relevan

```
environment:
  LAB_MODE: hardened
  DB_PASSWORD_FILE: /run/secrets/db_password
volumes:
  - ./runtime/secrets/runtime_current:/run/secrets/db_password:ro
```

Cuplikan di atas hanya menunjukkan bagian secret database. Mount session, CA, dan evidence tetap diperlukan sesuai compose.yaml.

```
password = Path(os.environ["DB_PASSWORD_FILE"]).read_text().strip()
```

Jangan menaruh password literal di kode atau menampilkan isi runtime/secrets. Nilai password role di database dan file secret yang dibaca aplikasi harus sesuai.

Praktik rotasi yang sudah disiapkan

```
docker compose --profile tools run --rm tester python tests/attacks.py --scenario S5
Get-Content .\evidence\attack-S5.json
```

Tahap otomatis	Hasil yang diharapkan
Password lama untuk baseline	Koneksi baru diterima.
Rotasi ke password saat ini	Password lama ditolak pada koneksi baru.
Gunakan password saat ini	Koneksi diterima; aplikasi hardened menghasilkan 200.

Jalankan berurutan

S5 sementara mengubah password role. Jangan jalankan bersama Test atau Attacks dan jangan hentikan di tengah proses. Blok finally memulihkan password saat ini pada penyelesaian atau kegagalan biasa. Sesi yang sudah tersambung tidak otomatis diputus oleh rotasi.

File secret mount ini mekanisme latihan, belum pengganti secret manager produksi. Jangan menghapus runtime secara terpisah dari volume database.

7 / Rapikan error dan log

app.py: database_error(), audit(), before() | Slide 35

Sebelum: detail exception database diberikan ke browser. **Sesudah:** browser menerima pesan umum dan request ID; log internal menyimpan kode error untuk penelusuran.

Perubahan respons error

```
# Baseline: detail error terlihat oleh pengguna
message = str(exc)

# Hardened: respons publik umum
audit("database_error", sqlstate=exc.sqlstate)
message = "Terjadi kesalahan internal"
return jsonify(error=message, request_id=g.request_id), 500
```

Di app.py aktual, pemilihan message menggunakan kondisi HARD. Request ID dibuat sebelum request dan dikirim juga lewat header X-Request-ID. Log tidak mencatat password atau teks pencarian mentah.

Praktik pembuktian

1. Login Alice pada 8083. Klik **Uji respons error**. Endpoint sengaja menjalankan SELECT 1 / 0.
2. Periksa HTTP 500, pesan umum, dan request ID. Catat ID dari respons atau header di Network browser.
3. Di PowerShell, jalankan dua baris berikut. Tempel hanya request ID ketika diminta, lalu Enter:

```
$requestId = Read-Host "Tempel request ID dari respons"
Select-String -Path .\evidence\hardened.jsonl -SimpleMatch $requestId
```

4. Cocokkan event database_error dan SQLSTATE **22012**. Request ID harus sama dengan kejadian yang baru diuji.

Apa yang dibuktikan?

Pengguna mendapat pesan yang cukup untuk melapor tanpa detail query internal. Tim teknis masih punya penghubung ke log. HTTP 500 memang diharapkan untuk endpoint latihan ini; bukan berarti hardening gagal.

Logging terstruktur sudah tersedia sejak baseline. Peningkatan pada tahap ini terutama membatasi detail publik. Untuk operasi nyata, log perlu retensi, pembatasan akses, pemantauan, dan pihak yang menangani alert.

8 / Buktikan pemulihan data

scripts/backup_restore.sh dan tests/restore_app.py | Slide 36

Hardening juga perlu didukung pemulihan yang diuji. Backup dibuat dengan pg_dump, dipulihkan ke database lain, lalu diperiksa dari sisi data dan aplikasi.

Jalankan dua langkah ini berurutan

```
docker compose exec -T db bash /lab-scripts/backup_restore.sh
docker compose --profile tools run --rm tester python tests/restore_app.py
```

Tunggu langkah pertama berhasil sebelum menjalankan langkah kedua. Script membuat database invoice_restore dengan penanda waktu; database utama tidak ditimpa.

Yang dikerjakan script

Langkah	Tujuan
pg_dump dengan format custom	Membuat dump database invoice_lab.
createdb lalu pg_restore	Memulihkan ke database lain pada cluster yang sama.
Atur CONNECT hasil restore	Hapus izin PUBLIC; berikan CONNECT ke app_runtime.
Bandingkan data	Target tiga baris dengan checksum sesuai sumber.
Uji aplikasi	Periksa fungsi dan otorisasi pada database hasil restore.

Baca bukti

```
Get-Content .\evidence\restore.json
Get-Content .\evidence\restore-app.json
```

1. Pada restore.json, periksa passed, nama database hasil restore, rows = 3, checksum, dan durasi aktual.
2. Pada restore-app.json, target empat pemeriksaan lulus. Aplikasi harus dapat memakai data hasil restore dengan batas akses yang benar.

Batas latihan

Ini bukan simulasi kehilangan seluruh cluster atau failover. Role yang sudah ada masih dipakai, dump tidak membawa global role/password, dan dump lokal belum dienkripsi. Setiap run menambah dump dan database restore; jangan menjalankan berulang tanpa kebutuhan.

9 / Retest hasil hardening

Slide 37-41 | Bukti penolakan harus disertai akses normal yang tetap berjalan

Jalankan rangkaian ini secara berurutan

```
.\lab.ps1 Test  
.\lab.ps1 Attacks
```

Tunggu Test selesai sebelum Attacks. Target: 51 pemeriksaan suite utama, 4 pemeriksaan aplikasi hasil restore, dan keluaran **FIVE SCENARIOS: 5/5**. Angka ini target lab, bukan jaminan hasil setiap run.

Kontrol	Bukti yang diharapkan
Binding	S1: payload tidak memperluas hasil pada 8082/8083; pencarian normal tetap bekerja.
Otorisasi objek	S2: Alice ditolak untuk INV-002 pada 8083; INV-001 tetap 200.
Least privilege	S3: akses internal dan UPDATE ditolak 42501; SELECT invoice tetap bisa.
Sumber koneksi	S4: .30 ditolak HBA; .40 bisa SELECT dengan akun valid.
Rotasi password	S5: password lama ditolak pada koneksi baru; password saat ini bekerja.
TLS dan error	CA/hostname salah dan plaintext ditolak; error publik umum, ID cocok di log.
Restore	Data sesuai dan empat pemeriksaan aplikasi hasil restore lulus.

Buka laporan terkini

```
Get-Content .\evidence\attacks.json  
Get-Content .\evidence\retest.json
```

Periksa timestamp, run ID, passed, total, serta actual tiap skenario. Laporan individual tidak otomatis menggantikan ringkasan attacks.json. Kegagalan sebelum agregasi bisa meninggalkan laporan ringkasan lama.

Jangan salah membaca PASS

Suite sengaja memeriksa bahwa baseline tetap rentan dan hardened menolak akses tertentu. PASS pada baseline berarti celah yang direncanakan berhasil direproduksi. Jika semua cabang diubah menjadi aman, ekspektasi baseline harus ditinjau; jangan menyembunyikan kegagalan tes.

Jika ingin mengedit sendiri

Pisahkan latihan implementasi dari paket pembanding yang dipakai saat webinar.

Menerapkan pola ini pada aplikasi sendiri

1. Simpan versi awal dan bekerja pada salinan source. Tuliskan hasil baseline: input yang membuka celah dan tindakan pengguna yang memang harus berhasil.
2. Terapkan perubahan secara bertahap: binding, otorisasi objek, akun runtime terbatas, TLS/sumber, lalu error dan pengelolaan secret. Uji kembali setelah setiap perubahan.
3. Buat migrasi role/grant yang sesuai database yang sudah ada. Mengedit `setup_lab.py` atau `init SQL` tidak mengubah role pada volume yang sudah terinisialisasi. Jangan reset volume hanya untuk menerapkan hak akses.

Jika mengubah `app.py` dalam paket latihan

Pertahankan cabang `vulnerable/query-fixed` bila ingin tetap memakai suite pembanding bawaan. Setelah perubahan kode, bangun ulang image dan buat ulang container aplikasi:

```
docker compose build setup
docker compose up -d --force-recreate vulnerable query-fixed hardened
```

Tunggu halaman login tersedia. Login ulang bila perlu, kemudian jalankan Test dan Attacks secara berurutan. Perintah ini bukan migrasi database; perubahan file HBA atau konfigurasi server juga memerlukan prosedur penerapan tersendiri.

Yang belum menjadikan lab siap produksi

Bagian	Kebutuhan lanjutan
Akses dan aplikasi	RLS bila sesuai arsitektur; aturan bisnis lengkap; autentikasi produksi; HTTPS browser dan cookie Secure.
Operasional	Secret manager, patching, pemantauan, pemilik respons, dan review perubahan.
Pemulihan	Backup terlindungi dan terpisah; retensi; uji RPO/RTO serta kegagalan infrastruktur.

Jangan menyalin pengecualian baseline, local trust, akun superuser aplikasi, atau server development Flask ke produksi. Kontrol lanjutan di tabel belum dibuktikan oleh lab.

Rujukan: `app.py`, `setup_lab.py`, `compose.yaml`, `db/pg_hba.conf`, `db/Dockerfile`, `db/start.sh`, `scripts/backup_restore.sh`, `tests/attacks.py`, `tests/retest.py`, dan panduan lab pada paket 26 September 2026. Potongan kode diatur ulang barisnya agar mudah dibaca. Perintah di dokumen belum dijalankan ulang saat PDF dibuat.